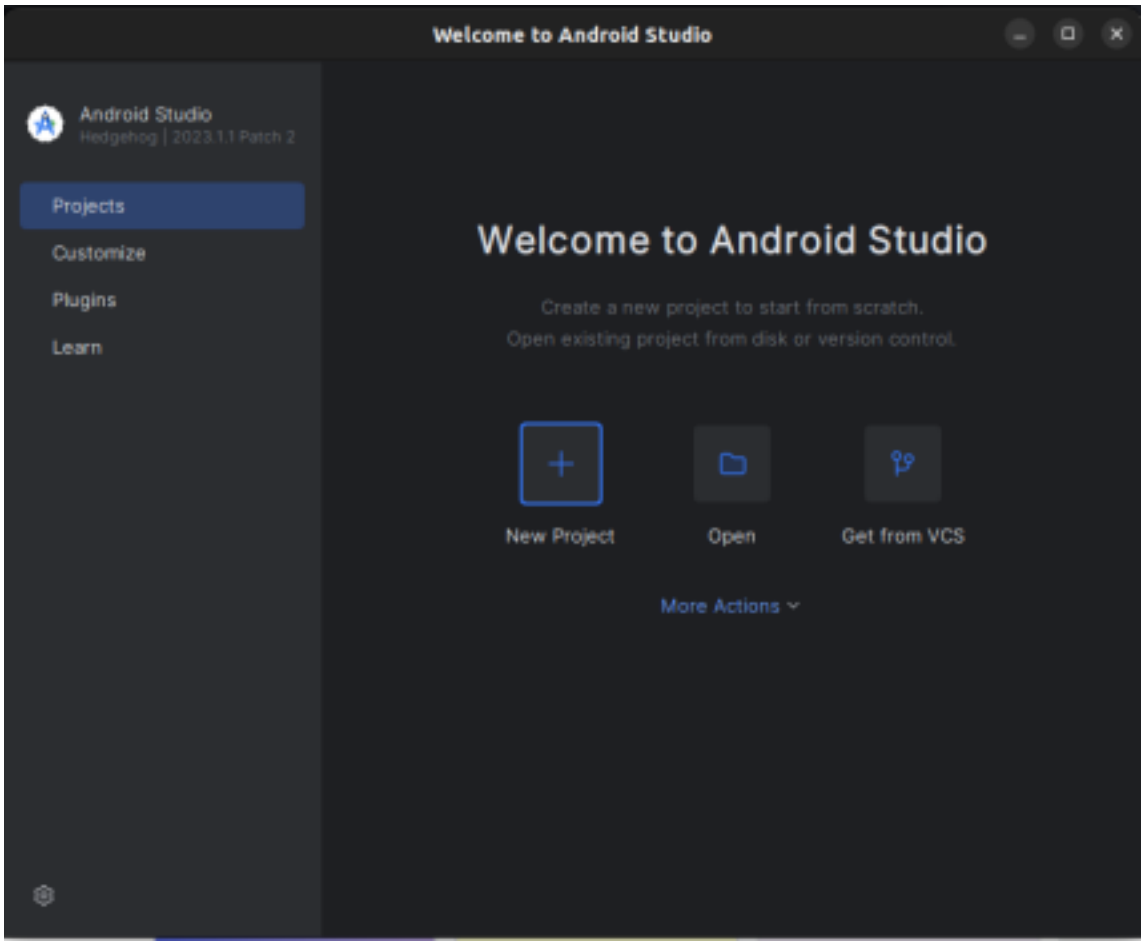
Laboratorijska vježba 1

Kreiranje prve aplikacije

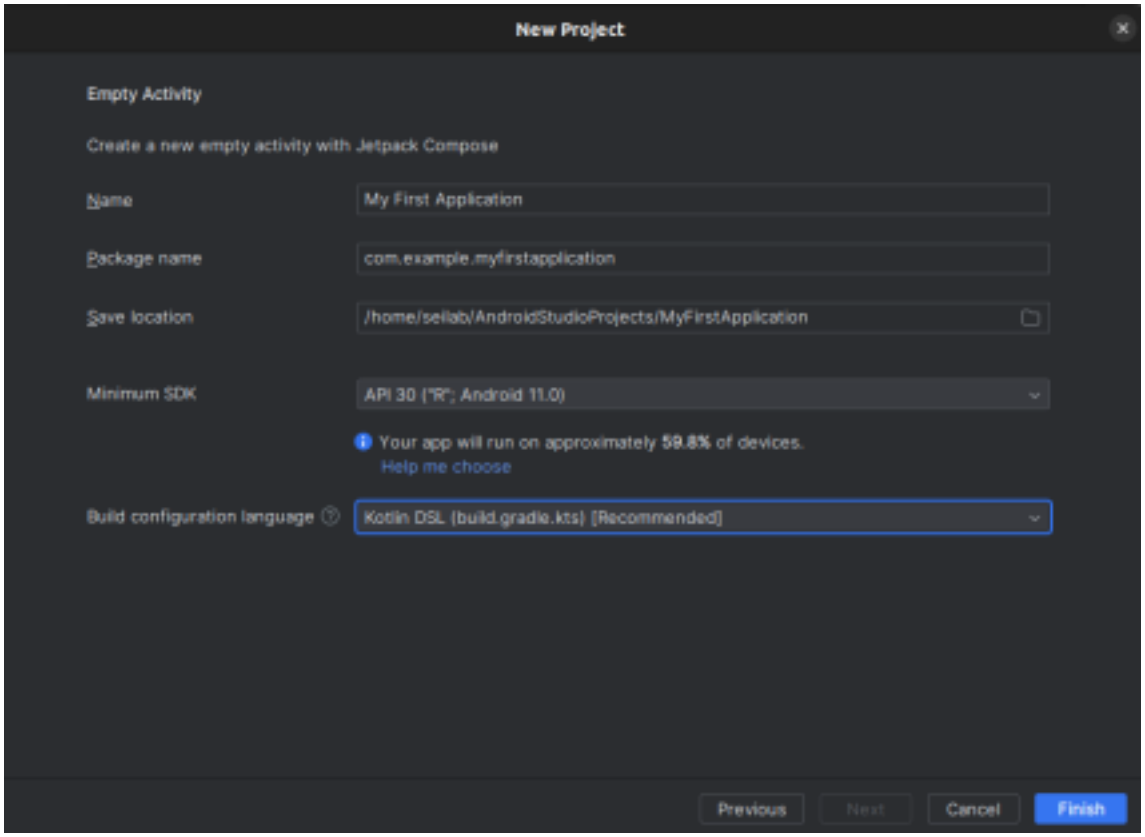
Zadatak 1. Kreiranje aplikacije

Kreirajte novi android projekat koristeći Android Studio:



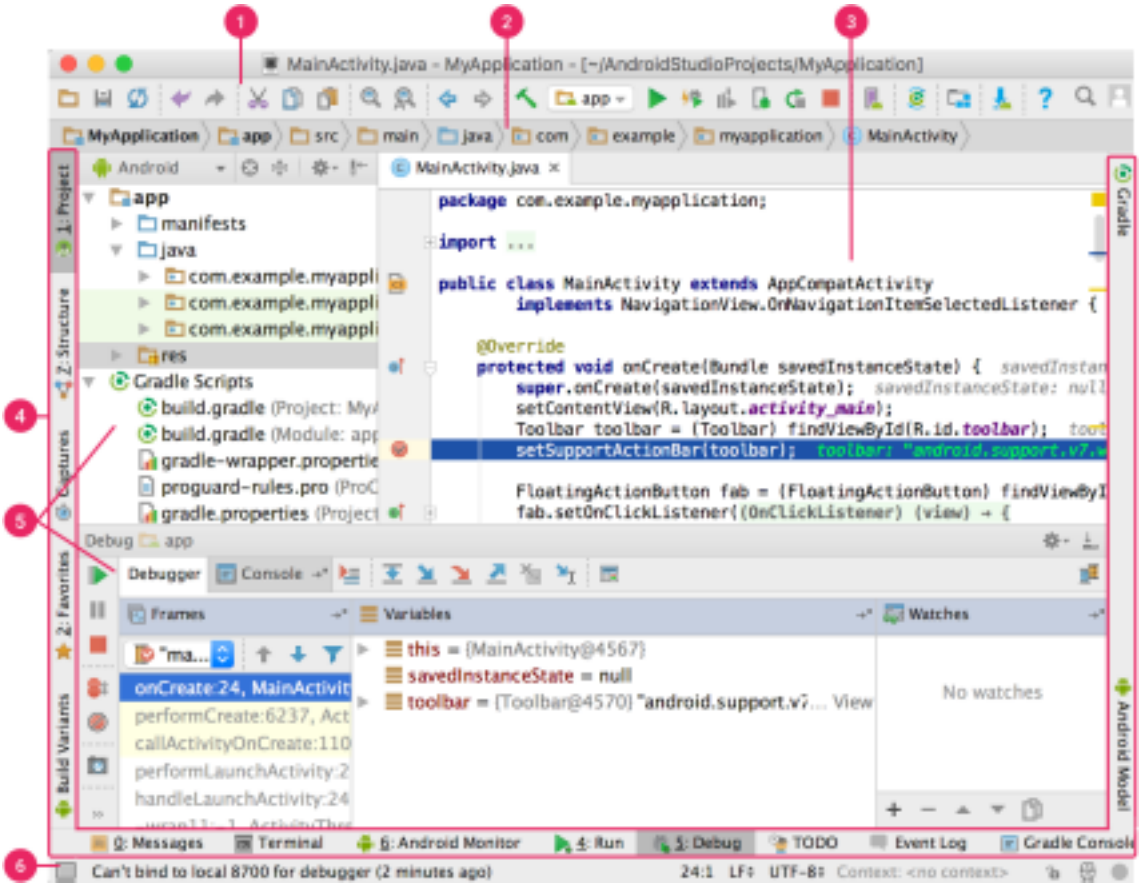
Izaberite Empty Views Activity :

Konfigurišite vaš projekat. Minimalni SDK postavite na API 30, ili niže na osnovu vlastitog uređaja:



Sačekajte da se kreira vaš projekat. Nakon kreiranja projekta isti će imati osnovne stavke kreirane: odgovarajuću strukturu sa manifest, glavnom aktivnošću i layout-om kreiranim.

Prođimo kroz osnovni prozor Android Studia.



U sklopu njega imamo sljedeće:

1. Toolbar koji omogućava izvršavanje raznih tipova akcija, kao što su pokretanje aplikacije i Android alata
2. Navigacijski bar omogućava prolaz kroz projekat i otvorene datoteke za uređivanje. Kompaktni pregled je dostupan u *Project* prozoru.
3. Editor služi za kreiranje i uređivanje kôda. Postoje dva tipa editor-a za kôd i za layout.
4. Toolbar omogućava prikazivanje određenih alata.
5. Alati projekta podrazumijevaju: menadžment projekta, pretraga, verzija, terminal, debug itd.
6. Statusna traka prikazuje trenutni status projekta i IDE.

Struktura android projekta je sljedeća:

manifests - Sadrži AndroidManifest.xml dokument.
kotlin+java - Sadrži izvorni kôd koji se razdvaja korištenjem paketa. Ovdje se nalaze i testovi
res - Sadrži sve resurse, kao što su XML layout-i, UI stringovi i slike koji su podijeljeni u odgovarajuće direktorije.

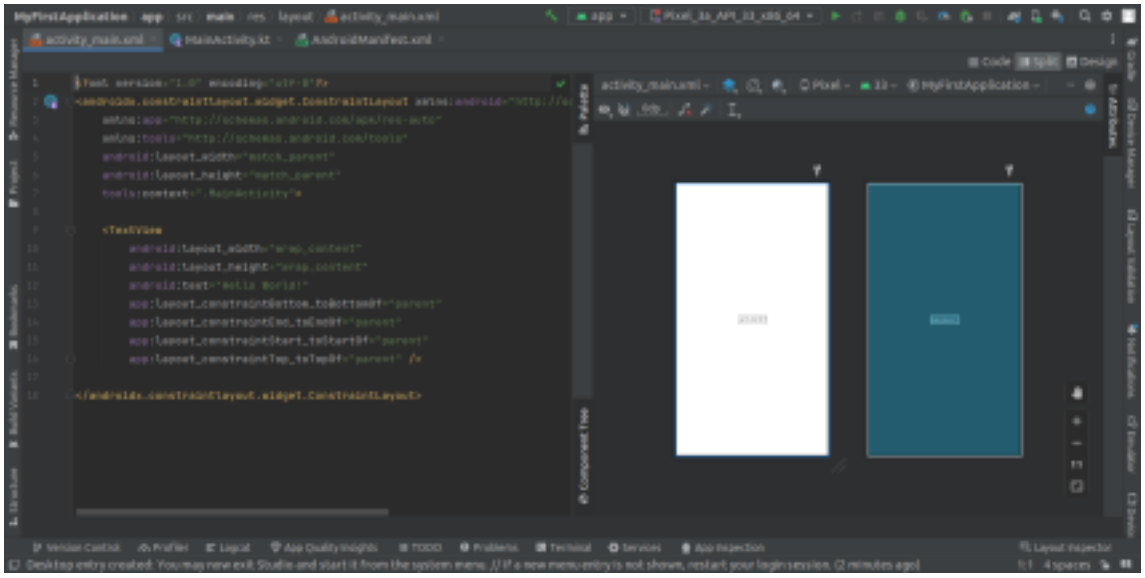
Struktura AndroidManifest.xml dokumenta je data u nastavku. U sklopu nje se definiše aplikacija i osnovne komponente android aplikacije.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="My First Application"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyFirstApplication"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

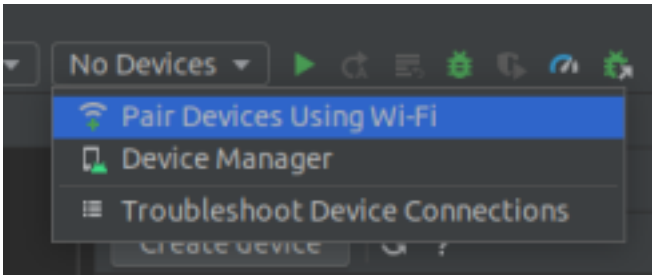
U nastavku je prikazan odgovarajući *LayoutEditor* koji ima dva vida prikaza: Uređivanje XML dokumenta i direktno uređivanje korisničkog interfejsa.



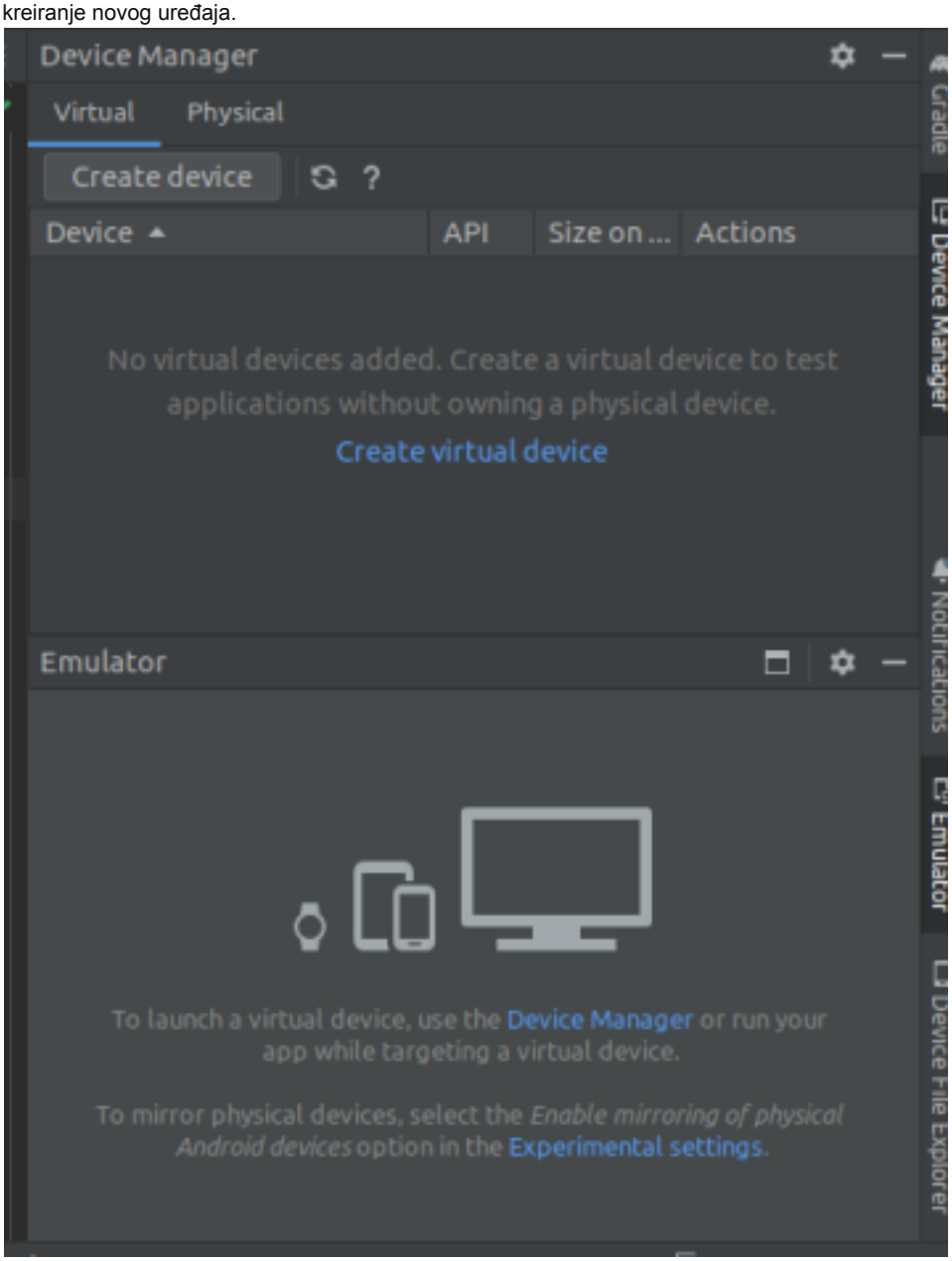
Zadatak 2. Pokretanje aplikacije

Pokrenimo sada našu aplikaciju. Aplikaciju možete pokretati na vlastitom uređaju pri čemu morate uključiti **USB DEBUGGING**. Da biste ovo uključili morate omogućiti razvojne
opcije (**Developer options**) klikom 7 puta na **VERSION NUMBER** u sklopu informacija o telefonu. Pored ovog načina, aplikaciju možete pokretati na emulatoru. U slučaju pokretanja na emulatoru, savjetuje se da se isti ne gasi, te da se samo aplikacija iznova pokreće na njemu. U nastavku slijede koraci instalacije jednog virtuelnog uređaja. **Napomena**: U BIOS-u opcije virtualizacije moraju biti uključene.

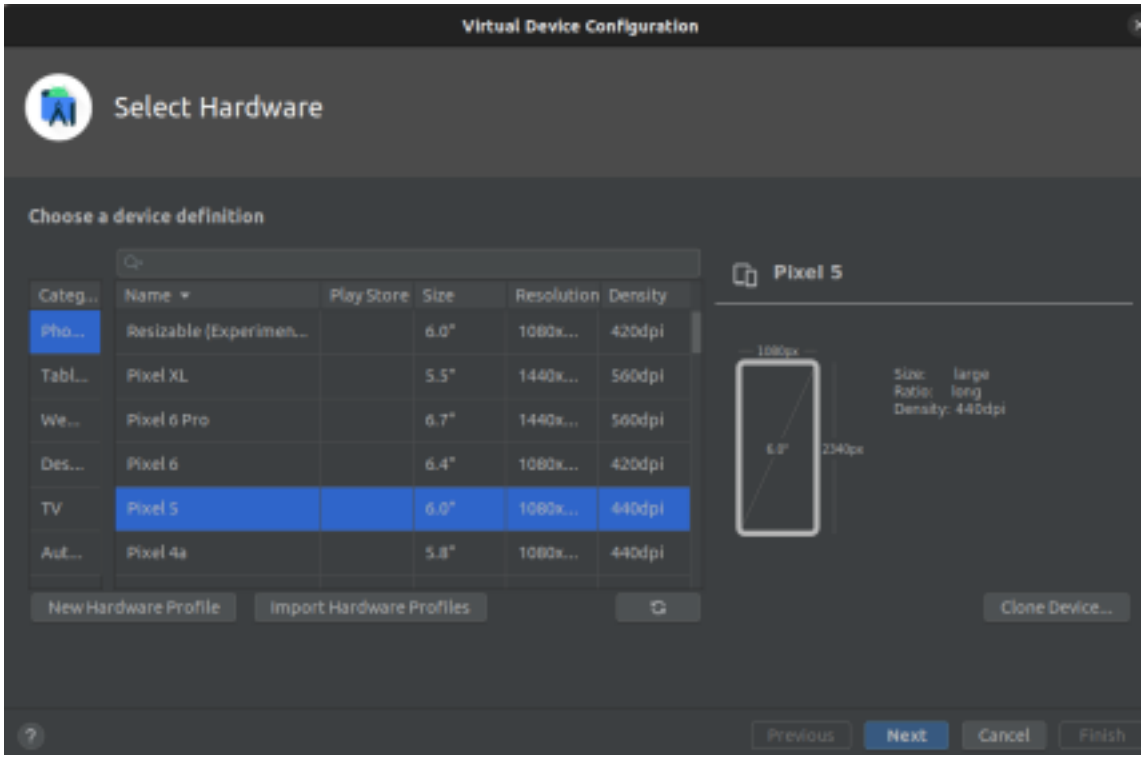
1. U sklopu Devices opcije pokrenite Device manager i odaberite opciju za



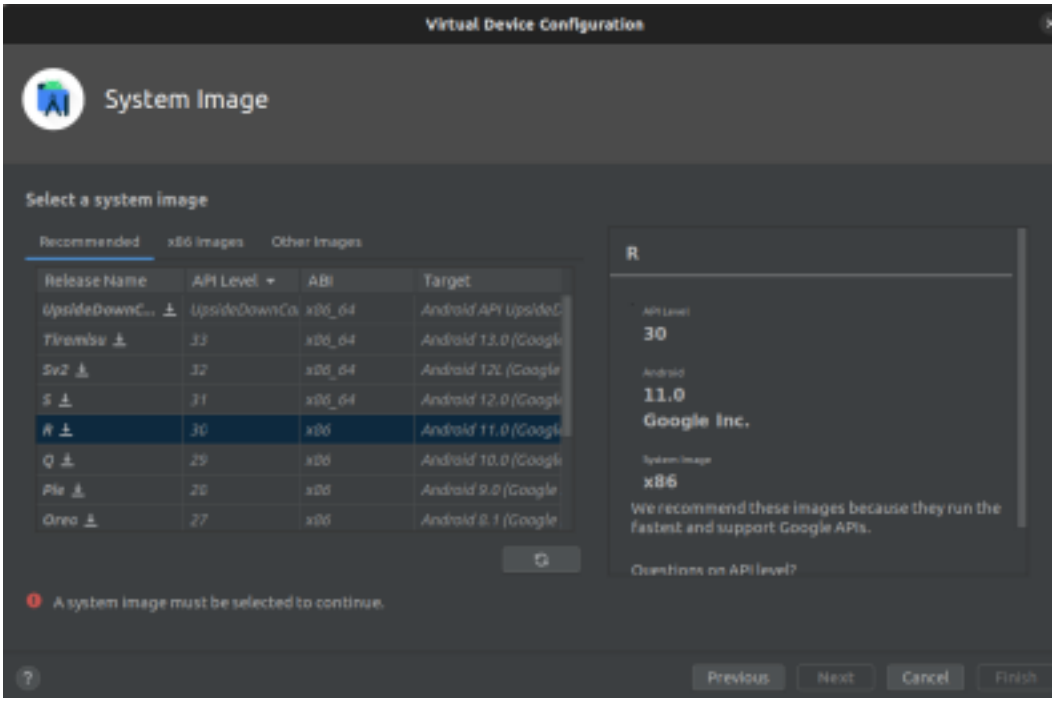
kreiranje novog uređaja.



2. Odaberite odgovarajući model uređaja.



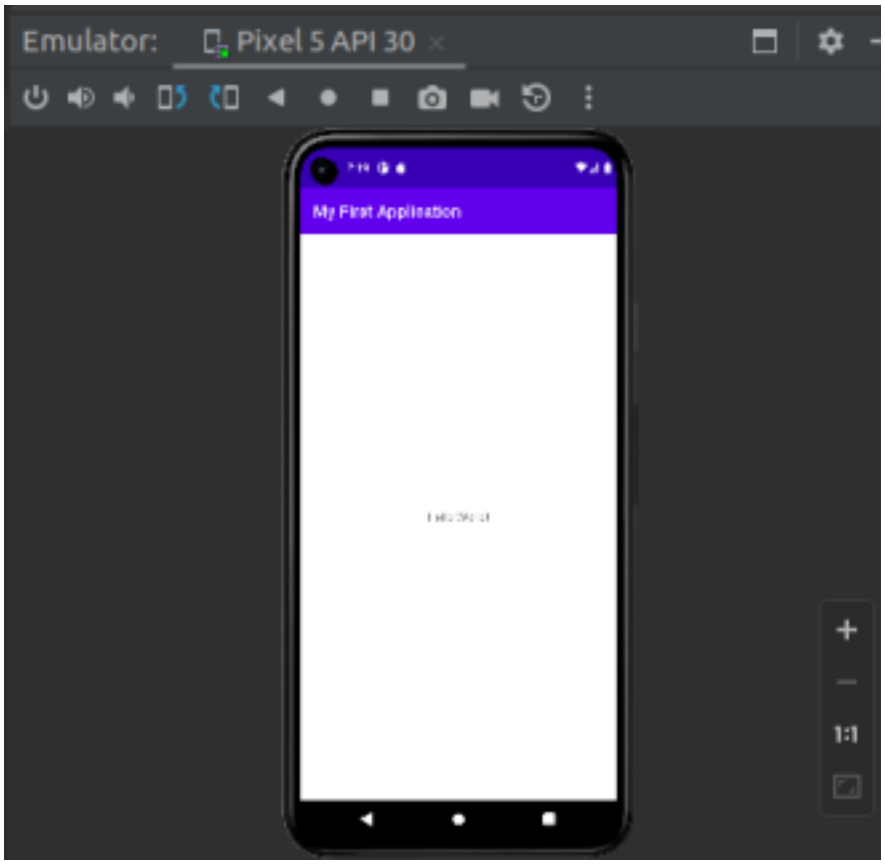
3. Odaberite operativni sistem (preporučuje se Android 11.0)



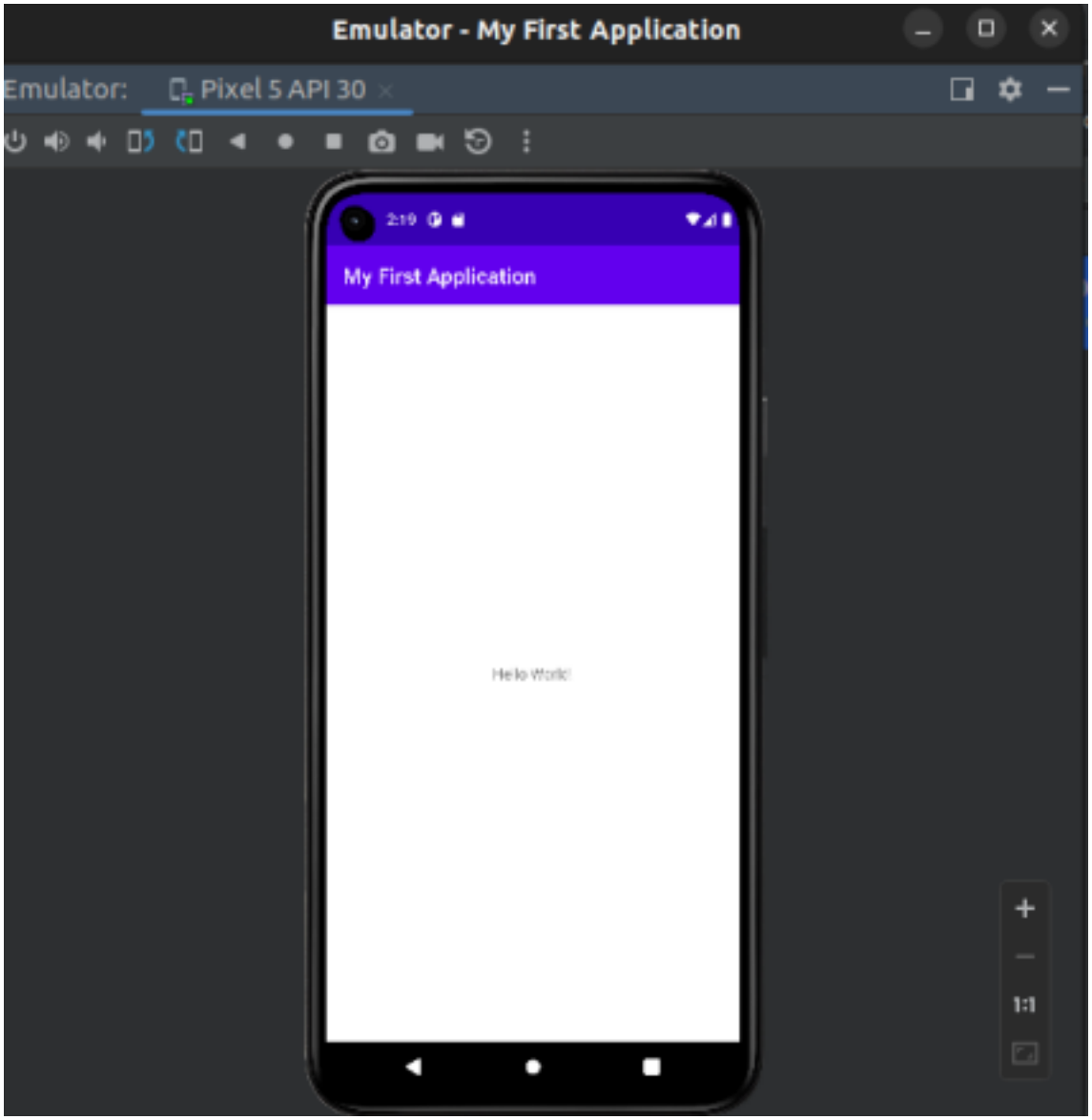
4.

5. Sačekajte da se instalacija završi.

Nakon što se instalacija završi, pokretanje aplikacije treba biti omogućeno. Pokrenite je na odgovarajućem uređaju. Emulator će se pokrenuti i nakon određenog vremena će se aplikacija build-ati i instalirati na uređaj. Emulator se trenutno nalazi kao dio Toolbar-a.



Ako želite odvojiti uređaj radi lakšeg pregleda, potrebno je da izaberete odgovarajuću opciju u sklopu **Settings-a/View Mode**.



Uspješno je pokrenuta prva aplikacija.

Zadatak 3. Korištenje ulaznih kontrola

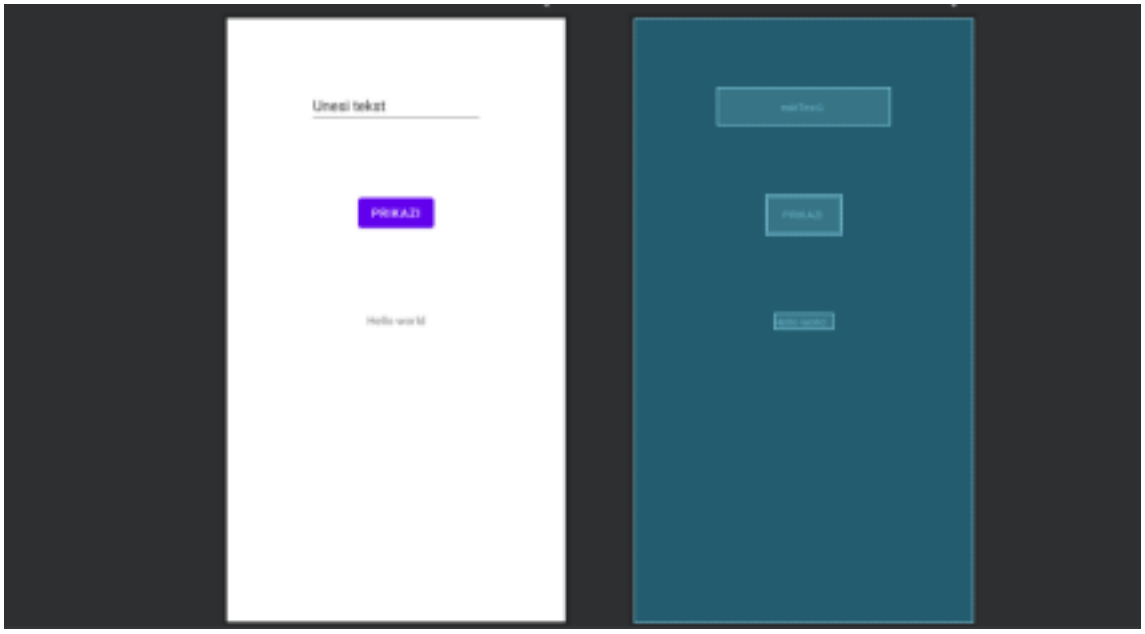
Potrebno je omogućiti unos teksta koji će s klikom na dugme biti prikaz u odgovarajućem TextView-u.

Korištenjem editora kreirat ćemo dodatne dvije kontrole:

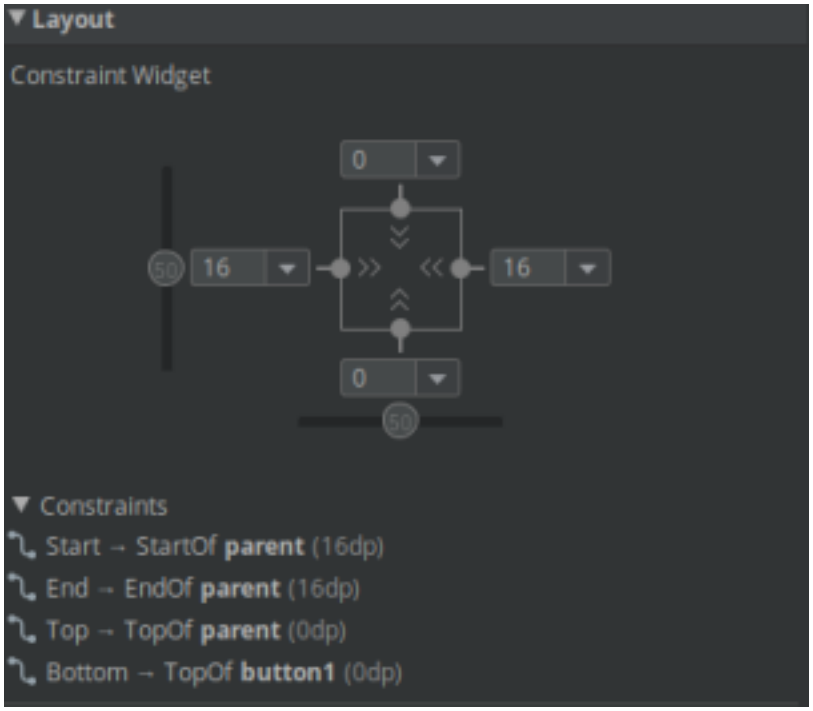
EditText s ID: `editText1`
Button s ID: button1

Postojeći TextView će imati ID: `textView1` .

Prikaz odgovarajućeg korisničkog interfejsa je dat na slici.



Napomena: Obzirom da je default layout ConstraintLayout za sve stavke definišite odgovarajuća ograničenja. Prikaz definisanja layout-a je dat na sljedećoj slici.



Nakon što smo kreirali komponente potrebno je da omogućimo izvršenje traženog kroz kôda.

Trenutno klasa glavne aktivnosti glasi:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Ovaj kôd označava da prilikom kreiranja aktivnosti će se prikazati odgovarajući layout activity_main . Layout-u se pristupa kroz klasu R koja označava sve interne resurse aplikacije. Resursi android-a su dostupni kroz android.R .

U sklopu odgovarajuće funkcije ćemo dohvatiti referencu na dugme i dodijeliti mu osluškivač na klik.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        //Dohvatit cemo referencu na view button-a preko id-a
        val button = findViewById<Button>(R.id.button1);
        //Definisat cemo akciju u slucaju klik akcije
        button.setOnClickListener {
            //akcije nakon klik-a
        }
    }
```

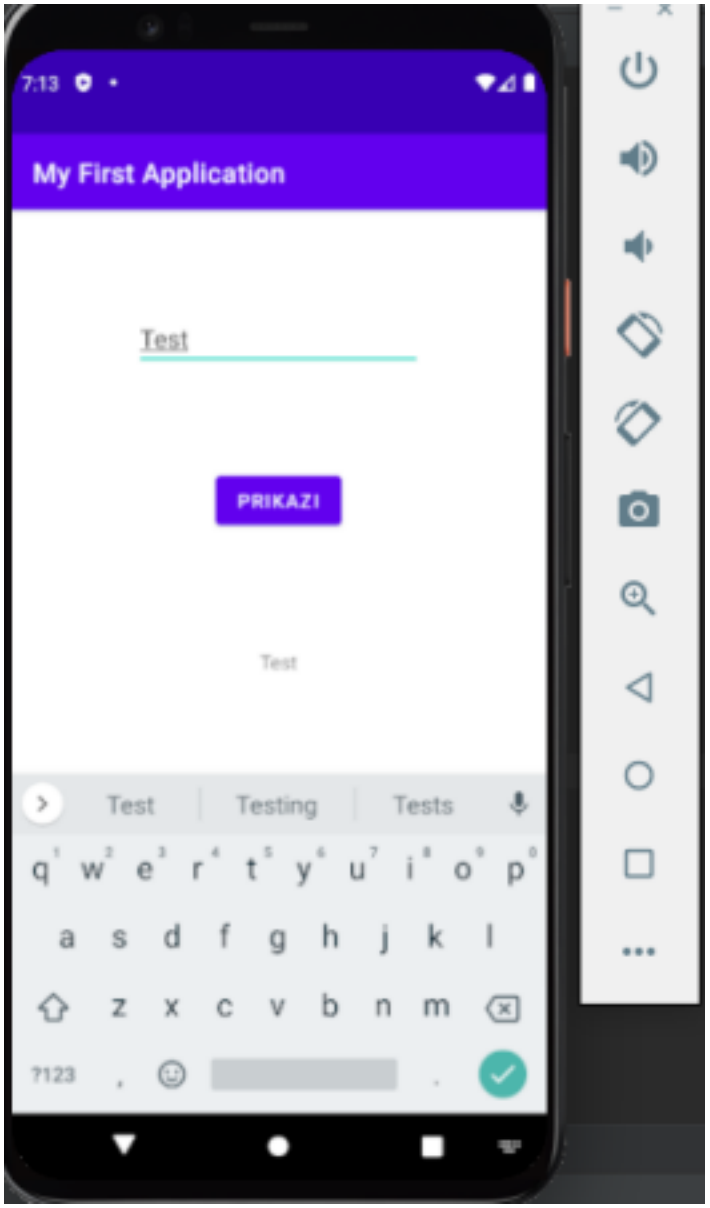
Kreirajmo privatnu funkciju unutar klase koja će dohvatiti tekst iz EditText -a i postaviti na TextView .

```
private fun showMessage() {
    // Pronaci cemo nas edit text i text view na osnovu id-a
    val editText = findViewById<EditText>(R.id.editText1)
    val textView = findViewById<TextView>(R.id.textView1)
    // Tekst cemo prebaciti u varijablu
    val message = editText.text.toString()
    // Postavimo tekst
    textView.text = message
}
```

Naš cjelokupni kôd sada izgleda ovako:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        //Dohvatit cemo referencu na view button-a preko id-a
        val button = findViewById<Button>(R.id.button1);
        //Definisat cemo akciju u slucaju klik akcije
        button.setOnClickListener {
            showMessage()
        }
    }
    private fun showMessage() {
        // Pronaci cemo nas edit text i text view na osnovu id-a
        val editText = findViewById<EditText>(R.id.editText1)
        val textView = findViewById<TextView>(R.id.textView1)
        // Tekst cemo prebaciti u varijablu
        val message = editText.text.toString()
        // Postavimo tekst
        textView.text = message
    }
}
```

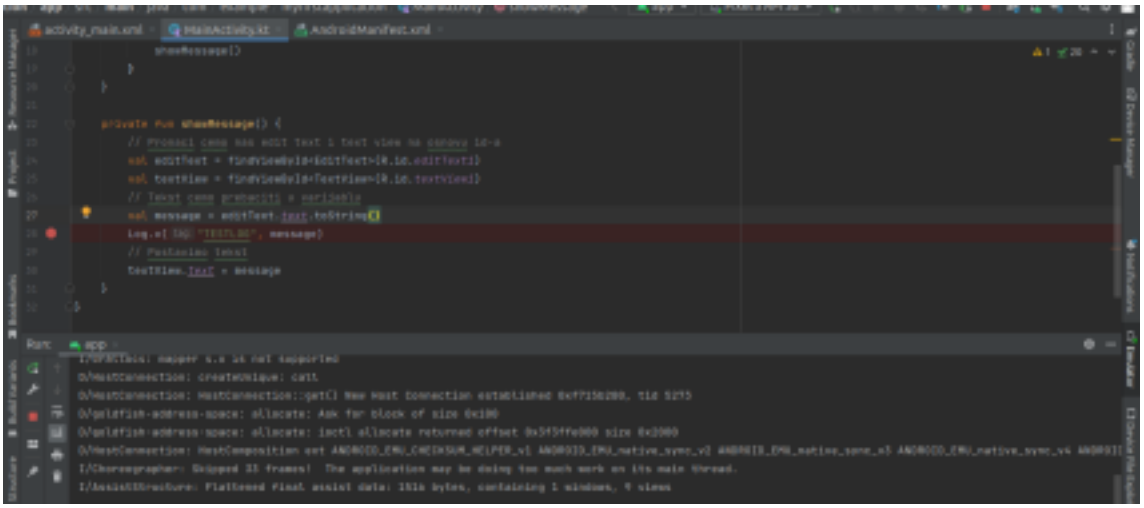
Nakon pokretanja i testiranja vidjet ćemo da aplikacija uradi traženo, kao što je prikazano na slici.



Zadatak 4: Debug aplikacije i logiranje

Postaviti breakpoint na aplikaciji i izvršiti logiranje teksta iz EditText-a.

U nastavku je prikazano kako se dodaje breakpoint, kako glasi odgovarajuća linija logiranja podataka i gdje se pokreće debug mode.



Nakon što se izvrše odgovarajuće akcije na aplikaciji, breakpoint će zaustaviti izvršavanje. Otvorit će se DEBUG tab u sklopu kojeg se vidi trenutačno stanje aplikacije. Izvršavanje se može nastaviti liniju po liniju ili do samog kraja.

Logiranje u sklopu ovog alata se vrši korištenjem **Logcat-a** Logovi se mogu vidjeti u sklopu odgovarajućeg Logcat tab-a. Postoje sljedeće vrste logova:

- Log.e(String, String) (error)
- Log.w(String, String) (warning)
- Log.i(String, String) (information)
- Log.d(String, String) (debug)
- Log.v(String, String) (verbose)

Prvi parametar predstavlja TAG, a drugi poruku. Prikaz odgovarajućeg loga aplikacije je dat na sljedećoj slici.

Omogućena je i detaljna pretraga logcat-a:

Zadatak 5

Projekat postaviti na vlastiti Bitbucket git repozitorij. Link na repozitorij trebate podijeliti s asistentima korištenjem njihovih fakultetskih e-mail adresa.

Za postavljanje projekta na git repozitorij možete koristiti VCS (Version Control) u sklopu Android Studia, ili možete koristiti komandnu liniju. Preporučuje se i upotreba GitKrakena radi bolje kontrole rada s git-om. U nastavku će biti prikazane osnovne naredbe.

Zadaci za samostalni rad

1. Pratite navedene zadatke.
2. Uredite aplikaciju da koristi LinearLayout, pa nakon toga i RelativeLayout. Koja je razlika između pojedinih layout-a?